

P1135R6

The C++20 Synchronization Library

Published Proposal, 2019-07-19

Authors:

[Bryce Adelstein Lelbach](#) (NVIDIA)

[Olivier Giroux](#) (NVIDIA)

[JF Bastien](#) (Apple)

[Detlef Vollmann](#)

[David Olsen](#) (NVIDIA)

Source:

[GitHub](#)

Issue Tracking:

[GitHub](#)

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Audience:

LWG

Table of Contents

1 Introduction

2 Changelog

3 Wording

Index

Terms defined by this specification

References

Informative References

§ 1. Introduction

This paper is the unification of a series of related C++20 proposals for introducing new synchronization and thread coordination facilities and enhancing existing ones:

- [\[P0514R4\]](#): Efficient `atomic` waiting and semaphores.
- [\[P0666R2\]](#): Latches and barriers.
- [\[P0995R1\]](#): `atomic_flag::test` and lockfree integral types.
- [\[P1258R0\]](#): Don't make C++ unimplementable for small CPUs.

The first part of this paper adds member functions `wait`, `notify_one`, and `notify_all` to `atomic<T>`. It does not add those same member functions to `atomic_ref<T>`, `atomic<shared_ptr<T>>`, or `atomic<weak_ptr<T>>`. Those omissions were partly an oversight, because the papers for those types were in flight at the same time as this paper. Adding `wait`, `notify_one`, and `notify_all` to the other types are not being added to this paper, but will be done separately in [\[P1643R0\]](#) and [\[P1644R0\]](#).

§ 2. Changelog

Revision 0: Post Rapperswil 2018 changes from [\[P0514R4\]](#), [\[P0666R2\]](#), and [\[P0995R1\]](#) based on [Rapperswil 2018 LEWG feedback](#).

- Refactored `basic_barrier` and `barrier` into one class with a default template parameter as suggested by LEWG at Rapperswil 2018.
- Refactored `basic_semaphore` and `counting_semaphore` into one class with a default template parameter as suggested by LEWG at Rapperswil 2018.
- Fixed `update` parameters in semaphore, latch, and barrier member functions to consistently default to 1 to resolve mistakes identified by LEWG at Rapperswil 2018.

Revision 1: Pre San Diego 2018 changes based on [Rapperswil 2018 LEWG feedback](#) and [a June discussion on the LEWG and SG1 mailing lists](#).

- Added member function versions of `atomic_wait_*` and `atomic_notify_*`, for consistency. Refactored wording to accommodate this.
- Renamed the `atomic_flag` overloads of `atomic_wait` and `atomic_wait_explicit` to `atomic_flag_wait` and `atomic_flag_wait_explicit` for consistency and to leave the door open for future compatibility with C.

- Renamed `latch::arrive_and_wait` and `barrier::arrive_and_wait` to `latch::sync` and `barrier::sync`, because LEWG at Rapperswil 2018 expected these methods to be the common use case and prefers they have a short name.
- Renamed `latch::arrive` to `latch::count_down` to further separate and distinguish the `latch` and `barrier` interfaces.
- Removed `barrier::try_wait` to resolve concerns raised during LEWG discussion at Rapperswil 2018 regarding its "maybe consuming" nature.
- Required that `barrier::arrival_token`'s move constructor and move assignment operators are `noexcept` to resolve discussions in LEWG at Rapperswil 2018 regarding exceptions being thrown when using the split arrive and wait barrier interface.
- Made `counting_semaphore::acquire`, `counting_semaphore::try_acquire`, and `latch::wait` `noexcept`, because participants in the mailing list discussion preferred that synchronization operations not throw and that any resource acquisition failures be reported by throwing during construction of synchronization objects.
- Made `counting_semaphore`, `latch`, and `barrier`'s constructors `non_constexpr` and allowed them to throw `system_error` if the object cannot be created, because participants in the mailing list discussion preferred that synchronization operations not throw and that any resource acquisition failures be reported by throwing during construction of synchronization objects.
- Clarified that `counting_semaphore::release`, `latch::count_down`, `latch::sync`, `barrier::wait`, `barrier::sync`, and `barrier::arrive_and_drop` throw nothing (but cannot be `noexcept`, because they have preconditions) to resolve discussions in LEWG at Rapperswil 2018 and on the mailing list.

Revision 2: San Diego 2018 changes to incorporate [\[P1258R0\]](#) and pre-meeting feedback.

- Made `barrier::wait` take its `arrival_token` parameter by rvalue reference.
- Made the `atomic_signed_lock_free` and `atomic_unsigned_lock_free` types optional for freestanding implementations, as per [\[P1258R0\]](#).

Revision 3: Pre Kona 2019 changes based on [San Diego 2018 LEWG feedback](#).

- Renamed `latch::sync` and `barrier::sync` back to `latch::arrive_and_wait` and `barrier::arrive_and_wait`, because this name had the strongest consensus in LEWG at San Diego 2018.
- Removed `atomic_int_fast_wait_t` and `atomic_uint_fast_wait_t`, because LEWG at San Diego 2018 felt that the use case was uncommon and the types had high potential for misuse.

- Made `counting_semaphore::acquire` and `latch::wait` non `noexcept` again, because LEWG at San Diego 2018 desired `constexpr` constructors for new synchronization objects to allow synchronization during program initialization and to maintain consistency with existing synchronization objects like `mutex`.
- Made `counting_semaphore`, `latch`, and `barrier`'s constructors `constexpr` again, because LEWG at San Diego 2018 desired `constexpr` constructors for new synchronization objects to allow synchronization during program initialization and to maintain consistency with existing synchronization objects like `mutex`.
- Clarified that `counting_semaphore::release`, `latch::count_down`, `latch::arrive_and_wait`, `barrier::wait`, `barrier::arrive_and_wait`, and `barrier::arrive_and_drop` may throw `system_error` exceptions, which is an implication of the constructors of said objects being `constexpr` because any underlying system errors must be reported on operations not during construction.
- Added missing `atomic<T>::wait` and `atomic<T>::notify_*` member functions to the class synopses for the `atomic<T>` integral, floating-point, and pointer specializations.
- Fixed `atomic<T>::notify_*` member functions to be non `const`.

Revision 4: Lots of wording changes based on [Kona 2019 LWG feedback](#). Three design changes to fix bugs that were discovered during LWG review or afterwards while revising the paper. These will be presented to SG1 in a separate paper, [\[P1633R0\]](#), in Cologne.

- Changed `atomic_flag::test` to be a `const` function. Changed the `atomic_flag*` parameter of `atomic_flag_test` and `atomic_flag_test_explicit` to be `const atomic_flag*`.
- Added the requirement that the `least_max_value` template parameter to `counting_semaphore` be greater than zero.
- Changed the requirement on the `update` parameter to `barrier::arrive` from `update >= 0` to `update > 0`.

Revision 5: Some wording improvements after the post-Kona mailing and before the pre-Cologne mailing. Incorporated feedback from LWG teleconferences on [7 June](#) and [14 June](#). Rebased the wording to be relative to the post-Kona draft, [\[N4810\]](#). There is one design change, which will be included in [\[P1633R0\]](#) along with the three changes in R4:

- Allow `latch::try_wait` to spuriously return `false`.

Revision 6: Removed the wording for the destructors of `counting_semaphore`, `latch`, and `barrier` that allowed the destructor to be called when some member functions in other threads had not yet returned. This design change has been added to [\[P1633R1\]](#). Based on SG1's discussion of

[P1633R1], the restriction on the template parameter of `counting_semaphore` was changed from `> 0` to `>= 0`. Several small wording improvements based on [LWG feedback](#) in Cologne.

§ 3. Wording

Note: The following changes are relative to the post Kona 2019 working draft of ISO/IEC 14882, ([N4810]).

Note: The  character is used to denote a placeholder number which shall be selected by the editor.

Add `<semaphore>`, `<latch>`, and `<barrier>` to [Table 19](#) "C++ library headers" in [\[headers\]](#).

Modify paragraph 3 in [16.5.1.3 \[compliance\]](#) as follows:

The supplied version of the header `<cstdlib>` shall declare at least the functions `abort`, `atexit`, `at_quick_exit`, `exit`, and `quick_exit` ([17.5](#)). The supplied version of the header `<atomic>` shall meet the same requirements as for a hosted implementation except that support for always lock-free integral atomic types ([\[atomics.lockfree\]](#)) is implementation-defined, and whether or not the type aliases `atomic_signed_lock_free` and `atomic_unsigned_lock_free` are defined ([\[atomics.alias\]](#)) is implementation-defined. The other headers listed in this table shall meet the same requirements as for a hosted implementation.

Modify the header synopsis for `<atomic>` in [\[atomics.syn\]](#) as follows:


```

namespace std {
    // ...

    // 31.8, non-member functions
    // ...

```

```

template<class T>
    void atomic_wait(const volatile atomic<T>*,
                    typename atomic<T>::value_type);
template<class T>
    void atomic_wait(const atomic<T>*,
                    typename atomic<T>::value_type);
template<class T>
    void atomic_wait_explicit(const volatile atomic<T>*,
                             typename atomic<T>::value_type,
                             memory_order);
template<class T>
    void atomic_wait_explicit(const atomic<T>*,
                             typename atomic<T>::value_type,
                             memory_order);
template<class T>
    void atomic_notify_one(volatile atomic<T>*);
template<class T>
    void atomic_notify_one(atomic<T>*);
template<class T>
    void atomic_notify_all(volatile atomic<T>*);
template<class T>
    void atomic_notify_all(atomic<T>*);

```

```

// 31.3, type aliases
// ...

```

```

using atomic_intptr_t      = atomic<intptr_t>;
using atomic_uintptr_t    = atomic<uintptr_t>;
using atomic_size_t       = atomic<size_t>;
using atomic_ptrdiff_t    = atomic<ptrdiff_t>;
using atomic_intmax_t      = atomic<intmax_t>;
using atomic_uintmax_t    = atomic<uintmax_t>;

```

```

using atomic_signed_lock_free = see below;
using atomic_unsigned_lock_free = see below;

```

```

// 31.9, flag type and operations

```

```
struct atomic_flag;
```

```
bool atomic_flag_test(const volatile atomic_flag*) noexcept;  
bool atomic_flag_test(const atomic_flag*) noexcept;  
bool atomic_flag_test_explicit(const volatile atomic_flag*, memory_order_order_t) noexcept;  
bool atomic_flag_test_explicit(const atomic_flag*, memory_order_order_t) noexcept;  
bool atomic_flag_test_and_set(volatile atomic_flag*) noexcept;  
bool atomic_flag_test_and_set(atomic_flag*) noexcept;  
bool atomic_flag_test_and_set_explicit(volatile atomic_flag*, memory_order_order_t) noexcept;  
bool atomic_flag_test_and_set_explicit(atomic_flag*, memory_order_order_t) noexcept;  
void atomic_flag_clear(volatile atomic_flag*) noexcept;  
void atomic_flag_clear(atomic_flag*) noexcept;  
void atomic_flag_clear_explicit(volatile atomic_flag*, memory_order_order_t) noexcept;  
void atomic_flag_clear_explicit(atomic_flag*, memory_order_order_t) noexcept;  
void atomic_flag_wait(const volatile atomic_flag*, bool) noexcept;  
void atomic_flag_wait(const atomic_flag*, bool) noexcept;  
void atomic_flag_wait_explicit(const volatile atomic_flag*,  
                               bool, memory_order_order_t) noexcept;  
void atomic_flag_wait_explicit(const atomic_flag*,  
                               bool, memory_order_order_t) noexcept;  
void atomic_flag_notify_one(volatile atomic_flag*) noexcept;  
void atomic_flag_notify_one(atomic_flag*) noexcept;  
void atomic_flag_notify_all(volatile atomic_flag*) const noexcept;  
void atomic_flag_notify_all(atomic_flag*) const noexcept;
```

```
#define ATOMIC_FLAG_INIT see below
```

```
// 30.10, fences
```

```
extern "C" void atomic_thread_fence(memory_order_order_t) noexcept;
```

```
extern "C" void atomic_signal_fence(memory_order_order_t) noexcept;
```

```
}
```

Modify [\[atomics.alias\]](#) as follows, adding a new paragraph to the end:

31.3 Type aliases

[**atomics.alias**]

- 1 The type aliases `atomic_intN_t`, `atomic_uintN_t`, `atomic_intptr_t`, and `atomic_uintptr_t` are defined if and only if `intN_t`, `uintN_t`, `intptr_t`, and `uintptr_t` are defined, respectively.
- 2 The type aliases `atomic_signed_lock_free` and `atomic_unsigned_lock_free` name specializations of `atomic` whose template arguments are integral types, respectively signed and unsigned, and whose `is_always_lock_free` property is `true`. [*Note*: These aliases are optional in freestanding implementations ([\[compliance\]](#)). - *end note*] Implementations should choose for these aliases the integral specializations of `atomic` for which the atomic waiting and notifying operations (**`atomics.wait`**) are most efficient.

Insert a new paragraph after paragraph 1 in [\[atomics.lockfree\]](#):

At least one signed integral specialization of the `atomic` template, along with the specialization for the corresponding unsigned type ([\[basic.fundamental\]](#)), are always lock-free. [*Note*: This requirement is optional in freestanding implementations ([\[compliance\]](#)). - *end note*]

Add a new subclause after [\[atomics.lockfree\]](#) with the stable name [**`atomics.wait`**]:

31. ♦ Waiting and notifying

[[atomics.wait](#)]

- 1 **Atomic waiting operations** and **atomic notifying operations** provide a mechanism to wait for the value of an atomic object to change more efficiently than can be achieved with polling. Atomic waiting operations may block until they are unblocked by atomic notifying operations, according to each function's effects. [*Note*: Programs are not guaranteed to observe transient atomic values, an issue known as the A-B-A problem, resulting in continued blocking if a condition is only temporarily met. – *end note*]
- 2 [*Note*: The following functions are atomic waiting operations:
 - (2.1) — `atomic<T>::wait`.
 - (2.2) — `atomic_flag::wait`.
 - (2.3) — `atomic_wait` and `atomic_wait_explicit`.
 - (2.4) — `atomic_flag_wait` and `atomic_flag_wait_explicit`.- *end note*]
- 3 [*Note*: The following functions are atomic notifying operations:
 - (3.1) — `atomic<T>::notify_one` and `atomic<T>::notify_all`.
 - (3.2) — `atomic_flag::notify_one` and `atomic_flag::notify_all`.
 - (3.3) — `atomic_notify_one` and `atomic_notify_all`.
 - (3.4) — `atomic_flag_notify_one` and `atomic_flag_notify_all`.- *end note*]
- 4 A call to an atomic waiting operation on an atomic object M is **eligible to be unblocked** by a call to an atomic notifying operation on M if there exist side effects X and Y on M such that:
 - (4.1) — the atomic waiting operation has blocked after observing the result of X,
 - (4.2) — X precedes Y in the modification order of M, and
 - (4.3) — Y happens before the call to the atomic notifying operation.

Drafting note: Adding atomic waiting and notifying operations to `atomic_ref<T>` and to `atomic<shared_ptr<T>>` is done in separate papers, [\[P1643R0\]](#) and [\[P1644R0\]](#) respectively, not as part of this paper.

Modify [\[atomics.types.generic\]](#) as follows:


```

namespace std {
    template<class T> struct atomic {
        using value_type = T;
        static constexpr bool is_always_lock_free = implementation-defined;
        bool is_lock_free() const volatile noexcept;
        bool is_lock_free() const noexcept;
        void store(T, memory_order = memory_order::seq_cst) volatile noexcept;
        void store(T, memory_order = memory_order::seq_cst) noexcept;
        T load(memory_order = memory_order::seq_cst) const volatile noexcept;
        T load(memory_order = memory_order::seq_cst) const noexcept;
        operator T() const volatile noexcept;
        operator T() const noexcept;
        T exchange(T, memory_order = memory_order::seq_cst) volatile noexcept;
        T exchange(T, memory_order = memory_order::seq_cst) noexcept;
        bool compare_exchange_weak(T&, T, memory_order, memory_order) volatile;
        bool compare_exchange_weak(T&, T, memory_order, memory_order) noexcept;
        bool compare_exchange_strong(T&, T, memory_order, memory_order) volatile;
        bool compare_exchange_strong(T&, T, memory_order, memory_order) noexcept;
        bool compare_exchange_weak(T&, T,
                                    memory_order = memory_order::seq_cst) volatile;
        bool compare_exchange_weak(T&, T,
                                    memory_order = memory_order::seq_cst) noexcept;
        bool compare_exchange_strong(T&, T,
                                    memory_order = memory_order::seq_cst) volatile;
        bool compare_exchange_strong(T&, T,
                                    memory_order = memory_order::seq_cst) noexcept;

        void wait(T, memory_order = memory_order::seq_cst) const volatile noexcept;
        void wait(T, memory_order = memory_order::seq_cst) const noexcept;
        void notify_one() volatile noexcept;
        void notify_one() noexcept;
        void notify_all() volatile noexcept;
        void notify_all() noexcept;

        atomic() noexcept = default;
        constexpr atomic(T) noexcept;
        atomic(const atomic&) = delete;
        atomic& operator=(const atomic&) = delete;
        atomic& operator=(const atomic&) volatile = delete;
        T operator=(T) volatile noexcept;
        T operator=(T) noexcept;
    };
}

```

```
};  
}
```

Drafting note: The behavior of the non-member functions `atomic_wait`, `atomic_wait_explicit`, `atomic_notify_one`, and `atomic_notify_all` is already covered by [\[atomics.nonmembers\]](#). Only the behavior of the member functions needs to be listed here.

Add the following to the end of [\[atomics.types.operations\]](#):

```
void wait(T old, memory_order order = memory_order::seq_cst) const volatile;
void wait(T old, memory_order order = memory_order::seq_cst) const noexcept;
```

1 *Expects:* order is neither `memory_order::release` nor `memory_order::acq_rel`.

2 *Effects:* Repeatedly performs the following steps, in order:

(2.1) — Evaluates `load(order)` and compares its value representation for equality against that of `old`.

(2.2) — If they compare unequal, returns.

(2.3) — Blocks until it is unblocked by an atomic notifying operation or is unblocked spuriously.

3 *Remarks:* This function is an atomic waiting operation ([**atomics.wait**]).

```
void notify_one() volatile noexcept;
void notify_one() noexcept;
```

4 *Effects:* Unblocks the execution of at least one atomic waiting operation that is eligible to be unblocked ([**atomics.wait**]) by this call, if any such atomic waiting operations exist.

5 *Remarks:* This function is an atomic notifying operation ([**atomics.wait**]).

```
void notify_all() volatile noexcept;
void notify_all() noexcept;
```

6 *Effects:* Unblocks the execution of all atomic waiting operations that are eligible to be unblocked ([**atomics.wait**]) by this call.

7 *Remarks:* This function is an atomic notifying operation ([**atomics.wait**]).

Modify [\[atomics.types.int\] paragraph 1](#) as follows:

- 1 There are specializations of the `atomic` class template for the integral types `char`, `signed char`, `unsigned char`, `short`, `unsigned short`, `int`, `unsigned int`, `long`, `unsigned long`, `long long`, `unsigned long long`, `char8_t`, `char16_t`, `char32_t`, `wchar_t`, and any other types needed by the typedefs in the header `<cstdint>`. For each such type *integral*, the specialization `atomic<integral>` provides additional atomic operations appropriate to integral types. [*Note*: For the specialization `atomic<bool>`, see [31.7](#). — *end note*]

```

namespace std {
    template<> struct atomic<integral> {
        using value_type = integral;
        using difference_type = value_type;
        static constexpr bool is_always_lock_free = implementation-defined;
        bool is_lock_free() const volatile noexcept;
        bool is_lock_free() const noexcept;
        void store(integral, memory_order = memory_order::seq_cst) volatile r
        void store(integral, memory_order = memory_order::seq_cst) noexcept;
        integral load(memory_order = memory_order::seq_cst) const volatile nc
        integral load(memory_order = memory_order::seq_cst) const noexcept;
        operator integral() const volatile noexcept;
        operator integral() const noexcept;
        integral exchange(integral, memory_order = memory_order::seq_cst) vol
        integral exchange(integral, memory_order = memory_order::seq_cst) noe
        bool compare_exchange_weak(integral&, integral,
                                   memory_order, memory_order) volatile noexc
        bool compare_exchange_weak(integral&, integral,
                                   memory_order, memory_order) noexcept;
        bool compare_exchange_strong(integral&, integral,
                                     memory_order, memory_order) volatile noe
        bool compare_exchange_strong(integral&, integral,
                                     memory_order, memory_order) noexcept;
        bool compare_exchange_weak(integral&, integral,
                                   memory_order = memory_order::seq_cst) vola
        bool compare_exchange_weak(integral&, integral,
                                   memory_order = memory_order::seq_cst) noex
        bool compare_exchange_strong(integral&, integral,
                                     memory_order = memory_order::seq_cst) vc
        bool compare_exchange_strong(integral&, integral,
                                     memory_order = memory_order::seq_cst) nc

        void wait(integral, memory_order = memory_order::seq_cst) const volat
        void wait(integral, memory_order = memory_order::seq_cst) const noexc
        void notify_one() volatile noexcept;
        void notify_one() noexcept;
        void notify_all() volatile noexcept;
        void notify_all() noexcept;

        integral fetch_add(integral, memory_order = memory_order::seq_cst) vc
        integral fetch_add(integral, memory_order = memory_order::seq_cst) nc
        integral fetch_sub(integral, memory_order = memory_order::seq_cst) vc
        integral fetch_sub(integral, memory_order = memory_order::seq_cst) nc
        integral fetch_and(integral, memory_order = memory_order::seq_cst) vc

```



```

integral fetch_and(integral, memory_order = memory_order::seq_cst) no
integral fetch_or(integral, memory_order = memory_order::seq_cst) vol
integral fetch_or(integral, memory_order = memory_order::seq_cst) noe
integral fetch_xor(integral, memory_order = memory_order::seq_cst) vc
integral fetch_xor(integral, memory_order = memory_order::seq_cst) no

```

```

atomic() noexcept = default;
constexpr atomic(integral) noexcept;
atomic(const atomic&) = delete;
atomic& operator=(const atomic&) = delete;
atomic& operator=(const atomic&) volatile = delete;
integral operator=(integral) volatile noexcept;
integral operator=(integral) noexcept;

```

```

integral operator++(int) volatile noexcept;
integral operator++(int) noexcept;
integral operator--(int) volatile noexcept;
integral operator--(int) noexcept;
integral operator++() volatile noexcept;
integral operator++() noexcept;
integral operator--() volatile noexcept;
integral operator--() noexcept;
integral operator+=(integral) volatile noexcept;
integral operator+=(integral) noexcept;
integral operator-=(integral) volatile noexcept;
integral operator-=(integral) noexcept;
integral operator&=(integral) volatile noexcept;
integral operator&=(integral) noexcept;
integral operator|=(integral) volatile noexcept;
integral operator|=(integral) noexcept;
integral operator^=(integral) volatile noexcept;
integral operator^=(integral) noexcept;

```

```

};

```

```

}

```

Modify [\[atomics.types.float\] paragraph 1](#) as follows:

- 1 There are specializations of the `atomic` class template for the floating-point types `float`, `double`, and `long double`. For each such type *floating-point*, the specialization `atomic<floating-point>` provides additional atomic operations appropriate to floating-point types.

```

namespace std {
    template<> struct atomic<floating-point> {
        using value_type = floating-point;
        using difference_type = value_type;
        static constexpr bool is_always_lock_free = implementation-defined;
        bool is_lock_free() const volatile noexcept;
        bool is_lock_free() const noexcept;
        void store(floating-point, memory_order = memory_order::seq_cst) volatile;
        void store(floating-point, memory_order = memory_order::seq_cst) noexcept;
        floating-point load(memory_order = memory_order::seq_cst) const volatile;
        floating-point load(memory_order = memory_order::seq_cst) const noexcept;
        operator floating-point() const volatile noexcept;
        operator floating-point() const noexcept;
        floating-point exchange(floating-point,
                                memory_order = memory_order::seq_cst) volatile;
        floating-point exchange(floating-point,
                                memory_order = memory_order::seq_cst) noexcept;
        bool compare_exchange_weak(floating-point&, floating-point,
                                   memory_order, memory_order) volatile noexcept;
        bool compare_exchange_weak(floating-point&, floating-point,
                                   memory_order, memory_order) noexcept;
        bool compare_exchange_strong(floating-point&, floating-point,
                                      memory_order, memory_order) volatile noexcept;
        bool compare_exchange_strong(floating-point&, floating-point,
                                      memory_order, memory_order) noexcept;
        bool compare_exchange_weak(floating-point&, floating-point,
                                   memory_order = memory_order::seq_cst) volatile;
        bool compare_exchange_weak(floating-point&, floating-point,
                                   memory_order = memory_order::seq_cst) noexcept;
        bool compare_exchange_strong(floating-point&, floating-point,
                                      memory_order = memory_order::seq_cst) volatile;
        bool compare_exchange_strong(floating-point&, floating-point,
                                      memory_order = memory_order::seq_cst) noexcept;

        void wait(floating-point,
                  memory_order = memory_order::seq_cst) const volatile noexcept;
        void wait(floating-point,
                  memory_order = memory_order::seq_cst) const noexcept;
        void notify_one() volatile noexcept;
        void notify_one() noexcept;
        void notify_all() volatile noexcept;
        void notify_all() noexcept;

        floating-point fetch_add(floating-point,

```

```

        memory_order = memory_order::seq_cst) volatile
floating-point fetch_add(floating-point,
        memory_order = memory_order::seq_cst) noexcept
floating-point fetch_sub(floating-point,
        memory_order = memory_order::seq_cst) volatile
floating-point fetch_sub(floating-point,
        memory_order = memory_order::seq_cst) noexcept

atomic() noexcept = default;
constexpr atomic(floating-point) noexcept;
atomic(const atomic&) = delete;
atomic& operator=(const atomic&) = delete;
atomic& operator=(const atomic&) volatile = delete;
floating-point operator=(floating-point) volatile noexcept;
floating-point operator=(floating-point) noexcept;

floating-point operator+=(floating-point) volatile noexcept;
floating-point operator+=(floating-point) noexcept;
floating-point operator-=(floating-point) volatile noexcept;
floating-point operator-=(floating-point) noexcept;
};
}

```

Modify [\[atomics.types.pointer\] paragraph 1](#) as follows:


```

namespace std {
    template<class T> struct atomic<T*> {
        using value_type = T*;
        using difference_type = ptrdiff_t;
        static constexpr bool is_always_lock_free = implementation-defined;
        bool is_lock_free() const volatile noexcept;
        bool is_lock_free() const noexcept;
        void store(T*, memory_order = memory_order::seq_cst) volatile noexcept;
        void store(T*, memory_order = memory_order::seq_cst) noexcept;
        T* load(memory_order = memory_order::seq_cst) const volatile noexcept;
        T* load(memory_order = memory_order::seq_cst) const noexcept;
        operator T*() const volatile noexcept;
        operator T*() const noexcept;
        T* exchange(T*, memory_order = memory_order::seq_cst) volatile noexcept;
        T* exchange(T*, memory_order = memory_order::seq_cst) noexcept;
        bool compare_exchange_weak(T*&, T*, memory_order, memory_order) volatile;
        bool compare_exchange_weak(T*&, T*, memory_order, memory_order) noexcept;
        bool compare_exchange_strong(T*&, T*, memory_order, memory_order) volatile;
        bool compare_exchange_strong(T*&, T*, memory_order, memory_order) noexcept;
        bool compare_exchange_weak(T*&, T*,
                                   memory_order = memory_order::seq_cst) volatile;
        bool compare_exchange_weak(T*&, T*,
                                   memory_order = memory_order::seq_cst) noexcept;
        bool compare_exchange_strong(T*&, T*,
                                   memory_order = memory_order::seq_cst) volatile;
        bool compare_exchange_strong(T*&, T*,
                                   memory_order = memory_order::seq_cst) noexcept;

        void wait(T*, memory_order = memory_order::seq_cst) const volatile noexcept;
        void wait(T*, memory_order = memory_order::seq_cst) const noexcept;
        void notify_one() volatile noexcept;
        void notify_one() noexcept;
        void notify_all() volatile noexcept;
        void notify_all() noexcept;

        T* fetch_add(ptrdiff_t, memory_order = memory_order::seq_cst) volatile;
        T* fetch_add(ptrdiff_t, memory_order = memory_order::seq_cst) noexcept;
        T* fetch_sub(ptrdiff_t, memory_order = memory_order::seq_cst) volatile;
        T* fetch_sub(ptrdiff_t, memory_order = memory_order::seq_cst) noexcept;

        atomic() noexcept = default;
        constexpr atomic(T*) noexcept;
        atomic(const atomic&) = delete;
        atomic& operator=(const atomic&) = delete;
        atomic& operator=(const atomic&) volatile = delete;
    };
}

```

```

T* operator=(T*) volatile noexcept;
T* operator=(T*) noexcept;

T* operator++(int) volatile noexcept;
T* operator++(int) noexcept;
T* operator--(int) volatile noexcept;
T* operator--(int) noexcept;
T* operator++() volatile noexcept;
T* operator++() noexcept;
T* operator--() volatile noexcept;
T* operator--() noexcept;
T* operator+=(ptrdiff_t) volatile noexcept;
T* operator+=(ptrdiff_t) noexcept;
T* operator-=(ptrdiff_t) volatile noexcept;
T* operator-=(ptrdiff_t) noexcept;
};
}

```

- 1 There is a partial specialization of the `atomic` class template for pointers. Specializations of this partial specialization are standard-layout structs. They each have a trivial default constructor and a trivial destructor.

Modify [\[atomics.flag\]](#) as follows:


```

namespace std {
    struct atomic_flag {
        bool test(memory_order = memory_order::seq_cst) const volatile noexcept;
        bool test(memory_order = memory_order::seq_cst) const noexcept;
        bool test_and_set(memory_order = memory_order::seq_cst) volatile noexcept;
        bool test_and_set(memory_order = memory_order::seq_cst) noexcept;
        void clear(memory_order = memory_order::seq_cst) volatile noexcept;
        void clear(memory_order = memory_order::seq_cst) noexcept;

        void wait(bool, memory_order = memory_order::seq_cst) const volatile;
        void wait(bool, memory_order = memory_order::seq_cst) const noexcept;
        void notify_one() volatile noexcept;
        void notify_one() noexcept;
        void notify_all() volatile noexcept;
        void notify_all() noexcept;

        atomic_flag() noexcept = default;
        atomic_flag(const atomic_flag&) = delete;
        atomic_flag& operator=(const atomic_flag&) = delete;
        atomic_flag& operator=(const atomic_flag&) volatile = delete;
    };

    bool atomic_flag_test(const volatile atomic_flag*) noexcept;
    bool atomic_flag_test(const atomic_flag*) noexcept;
    bool atomic_flag_test_explicit(const volatile atomic_flag*, memory_order) noexcept;
    bool atomic_flag_test_explicit(const atomic_flag*, memory_order) noexcept;
    bool atomic_flag_test_and_set(volatile atomic_flag*) noexcept;
    bool atomic_flag_test_and_set(atomic_flag*) noexcept;
    bool atomic_flag_test_and_set_explicit(volatile atomic_flag*, memory_order) noexcept;
    bool atomic_flag_test_and_set_explicit(atomic_flag*, memory_order) noexcept;
    void atomic_flag_clear(volatile atomic_flag*) noexcept;
    void atomic_flag_clear(atomic_flag*) noexcept;
    void atomic_flag_clear_explicit(volatile atomic_flag*, memory_order) noexcept;
    void atomic_flag_clear_explicit(atomic_flag*, memory_order) noexcept;

    void atomic_flag_wait(const volatile atomic_flag*, bool) noexcept;
    void atomic_flag_wait(const atomic_flag*, bool) noexcept;
    void atomic_flag_wait_explicit(const volatile atomic_flag*,
                                   bool, memory_order) noexcept;
    void atomic_flag_wait_explicit(const atomic_flag*,
                                   bool, memory_order) noexcept;
    void atomic_flag_notify_one(volatile atomic_flag*) noexcept;
    void atomic_flag_notify_one(atomic_flag*) noexcept;

```

```
void atomic_flag_notify_all(volatile atomic_flag*) const noexcept;
void atomic_flag_notify_all(atomic_flag*) const noexcept;
```

```
#define ATOMIC_FLAG_INIT see below
}
```

- 1 The `atomic_flag` type provides the classic test-and-set functionality. It has two states, set and clear.
- 2 Operations on an object of type `atomic_flag` shall be lock-free. [*Note:* Hence the operations should also be address-free. — *end note*]
- 3 The `atomic_flag` type is a standard-layout struct. It has a trivial default constructor and a trivial destructor.
- 4 The macro `ATOMIC_FLAG_INIT` shall be defined in such a way that it can be used to initialize an object of type `atomic_flag` to the clear state. The macro can be used in the form:

```
atomic_flag guard = ATOMIC_FLAG_INIT;
```

It is unspecified whether the macro can be used in other initialization contexts. For a complete static-duration object, that initialization shall be static. Unless initialized with `ATOMIC_FLAG_INIT`, it is unspecified whether an `atomic_flag` object has an initial state of set or clear.

```
bool atomic_flag_test(const volatile atomic_flag* object) noexcept;
bool atomic_flag_test(const atomic_flag* object) noexcept;
bool atomic_flag_test_explicit(const volatile atomic_flag* object,
                               memory_order order) noexcept;
bool atomic_flag_test_explicit(const atomic_flag* object,
                               memory_order order) noexcept;
bool atomic_flag::test(memory_order order =
                      memory_order::seq_cst) const volatile noexcept;
bool atomic_flag::test(memory_order order =
                      memory_order::seq_cst) const noexcept;
```

- 5 For `atomic_flag_test`, let order be `memory_order::seq_cst`.
- 6 *Expects:* order is neither `memory_order::release` nor `memory_order::acq_rel`.
- 7 *Effects:* Memory is affected according to the value of order.

8 *Returns:* Atomically returns the value pointed to by object or this.

```
bool atomic_flag_test_and_set(volatile atomic_flag* object) noexcept;
bool atomic_flag_test_and_set(atomic_flag* object) noexcept;
bool atomic_flag_test_and_set_explicit(volatile atomic_flag* object,
                                       memory_order order) noexcept;
bool atomic_flag_test_and_set_explicit(atomic_flag* object,
                                       memory_order order) noexcept;
bool atomic_flag::test_and_set(memory_order order =
                               memory_order::seq_cst) volatile noexcept;
bool atomic_flag::test_and_set(memory_order order =
                               memory_order::seq_cst) noexcept;
```

9 *Effects:* Atomically sets the value pointed to by object or by this to true. Memory is affected according to the value of order. These operations are atomic read-modify-write operations (4.7).

10 *Returns:* Atomically, the value of the object immediately before the effects.

```
void atomic_flag_clear(volatile atomic_flag* object) noexcept;
void atomic_flag_clear(atomic_flag* object) noexcept;
void atomic_flag_clear_explicit(volatile atomic_flag* object,
                               memory_order order) noexcept;
void atomic_flag_clear_explicit(atomic_flag* object,
                               memory_order order) noexcept;
void atomic_flag::clear(memory_order order =
                       memory_order::seq_cst) volatile noexcept;
void atomic_flag::clear(memory_order order =
                       memory_order::seq_cst) noexcept;
```

Expects: order is neither memory_order::consume, memory_order::acquire, nor memory_order::acq_rel.

Effects: Atomically sets the value pointed to by object or by this to false. Memory is affected according to the value of order.

```

void atomic_flag_wait(const volatile atomic_flag* object, bool old) noexcept;
void atomic_flag_wait(const atomic_flag* object, bool old) noexcept;
void atomic_flag_wait_explicit(const volatile atomic_flag* object,
                               bool old, memory_order order) noexcept;
void atomic_flag_wait_explicit(const atomic_flag* object,
                               bool old, memory_order order) noexcept;
void atomic_flag::wait(bool old, memory_order order =
                      memory_order::seq_cst) const volatile noexcept;
void atomic_flag::wait(bool old, memory_order order =
                      memory_order::seq_cst) const noexcept;

```

11 For `atomic_flag_wait`, let `order` be `memory_order::seq_cst`. Let `flag` be `object` for the non-member functions and `this` for the member functions.

12 *Expects:* `order` is neither `memory_order::release` nor `memory_order::acq_rel`.

13 *Effects:* Repeatedly performs the following steps, in order:

(13.1) — Evaluates `flag->test(order) != old`.

(13.2) — If the result of that evaluation is `true`, returns.

(13.3) — Blocks until it is unblocked by an atomic notifying operation or is unblocked spuriously.

14 *Remarks:* This function is an atomic waiting operation ([**atomics.wait**]).

```

void atomic_flag_notify_one(volatile atomic_flag* object) noexcept;
void atomic_flag_notify_one(atomic_flag* object) noexcept;
void atomic_flag::notify_one() volatile noexcept;
void atomic_flag::notify_one() noexcept;

```

15 *Effects:* Unblocks the execution of at least one atomic waiting operation that is eligible to be unblocked ([**atomics.wait**]) by this call, if any such atomic waiting operations exist.

16 *Remarks:* This function is an atomic notifying operation ([**atomics.wait**]).

```

void atomic_flag_notify_all(volatile atomic_flag* object) const noexcept;
void atomic_flag_notify_all(atomic_flag* object) const noexcept;
void atomic_flag::notify_all() volatile noexcept;
void atomic_flag::notify_all() noexcept;

```

- 17 *Effects:* Unblocks the execution of all atomic waiting operations that are eligible to be unblocked ([**atomics.wait**]) by this call.
- 18 *Remarks:* This function is an atomic notifying operation ([**atomics.wait**]).

Modify Table 134 "Thread support library summary" in [\[thread.general\]](#) as follows:

Table 134 — Thread support library summary

	Subclause	Header(s)
32.2	Requirements	
32.3	Threads	<thread>
32.4	Mutual exclusion	<mutex> <shared_mutex>
32.5	Condition variables	<condition_variable>
32.◆	Semaphores	<semaphore>
32.◆	Latches and barriers	<latch> <barrier>
32.6	Futures	<future>

Add two new subclauses after [\[thread.condition\]](#):

32.◆ Semaphores

[thread.semaphore]

- 1 Semaphores are lightweight synchronization primitives used to constrain concurrent access to a shared resource. They are widely used to implement other synchronization primitives and, whenever both are applicable, can be more efficient than condition variables.
- 2 A counting semaphore is a semaphore object that models a non-negative resource count. A binary semaphore is a semaphore object that has only two states. A binary semaphore should be more efficient than the default implementation of a counting semaphore with a unit resource count.

32.◆.1 Header <semaphore> synopsis

[thread.semaphore.syn]

```
namespace std {
    template<ptrdiff_t least_max_value = implementation-defined>
        class counting_semaphore;

    using binary_semaphore = counting_semaphore<1>;
}
```

32.2 Class template `counting_semaphore`

[`thread.semaphore.counting.class`]

```
namespace std {
    template<ptrdiff_t least_max_value = implementation-defined>
    class counting_semaphore {
    public:
        static constexpr ptrdiff_t max() noexcept;

        constexpr explicit counting_semaphore(ptrdiff_t desired);
        ~counting_semaphore();

        counting_semaphore(const counting_semaphore&) = delete;
        counting_semaphore& operator=(const counting_semaphore&) = delete;

        void release(ptrdiff_t update = 1);
        void acquire();
        bool try_acquire() noexcept;
        template<class Rep, class Period>
            bool try_acquire_for(const chrono::duration<Rep, Period>& rel_time)
        template<class Clock, class Duration>
            bool try_acquire_until(const chrono::time_point<Clock, Duration>& a

    private:
        ptrdiff_t counter; // exposition only
    };
}
```

- 1 Class `counting_semaphore` maintains an internal counter that is initialized when the semaphore is created. The counter is decremented when a thread acquires the semaphore, and is incremented when a thread releases the semaphore. If a thread tries to acquire the semaphore when the counter is zero, the thread will block until another thread increments the counter by releasing the semaphore.
- 2 `least_max_value` shall be non-negative; otherwise the program is ill-formed.
- 3 Concurrent invocations of the member functions of `counting_semaphore`, other than its destructor, do not introduce data races.

```
static constexpr ptrdiff_t max() noexcept;
```

- 4 *Returns:* The maximum value of counter. This value is greater than or equal to `least_max_value`.

```
constexpr explicit counting_semaphore(ptrdiff_t desired);
```

5 *Expects:* `desired >= 0` is true, and `desired <= max()` is true.

6 *Effects:* Initializes counter with `desired`.

7 *Throws:* Nothing.

```
void release(ptrdiff_t update = 1);
```

8 *Expects:* `update >= 0` is true, and `update <= max() - counter` is true.

9 *Effects:* Atomically execute `counter += update`. Then, unblock any threads that are waiting for counter to be greater than zero.

10 *Synchronization:* Strongly happens before invocations of `try_acquire` that observe the result of the effects.

11 *Throws:* `system_error` when an exception is required ([\[thread.req.exception\]](#)).

12 *Error conditions:* Any of the error conditions allowed for mutex types ([\[thread.mutex.requirements.mutex\]](#)).

```
bool try_acquire() noexcept;
```

13 *Effects:*

(13.1) — With low probability, returns immediately. An implementation should ensure that `try_acquire` does not consistently return `false` in the absence of contending acquisitions.

(13.2) — Otherwise, atomically check whether counter is greater than zero and, if so, decrement counter by one.

14 *Returns:* `true` if counter was decremented, otherwise `false`.

```
void acquire();
```

15 *Effects:* Repeatedly performs the following steps, in order:

(15.1) — Evaluates `try_acquire`. If the result is `true`, returns.

(15.2) — Blocks on `*this` until counter is greater than zero.

16 *Throws:* `system_error` when an exception is required ([\[thread.req.exception\]](#)).

17 *Error conditions:* Any of the error conditions allowed for mutex types ([\[thread.mutex.requirements.mutex\]](#)).

```
template<class Rep, class Period>
    bool try_acquire_for(const chrono::duration<Rep, Period>& rel_time);
template<class Clock, class Duration>
    bool try_acquire_until(const chrono::time_point<Clock, Duration>& abs_t
```

18 *Effects:* Repeatedly performs the following steps, in order:

(18.1) — Evaluates `try_acquire`. If the result is `true`, returns `true`.

(18.2) — Blocks on `*this` until `counter` is greater than zero or until the timeout expires. If it is unblocked by the timeout expiring, returns `false`.

The timeout expires ([\[thread.req.timing\]](#)) when the current time is after `abs_time` (for `try_acquire_until`) or when at least `rel_time` has passed from the start of the function (for `try_acquire_for`).

19 *Throws:* Timeout-related exceptions ([\[thread.req.timing\]](#)), or `system_error` when a non-timeout-related exception is required ([\[thread.req.exception\]](#)).

20 *Error conditions:* Any of the error conditions allowed for mutex types ([\[thread.mutex.requirements.mutex\]](#)).

32.◆ Coordination Types

[thread.coord]

1 This subclause describes various concepts related to thread coordination, and defines the coordination types `latch` and `barrier`. These types facilitate concurrent computation performed by a number of threads.

32.◆.1 Latches

[thread.coord.latch]

1 A latch is a thread coordination mechanism that allows any number of threads to block until an expected number of threads arrive at the latch (via the `count_down` function). The expected count is set when the latch is created. An individual latch is a single-use object; once the expected count has been reached, the latch cannot be reused.

32.1.1 Header <latch> synopsis

[thread.coord.latch.syn]

```
namespace std {  
    class latch;  
}
```

32.1.2 Class `latch`

[`thread.coord.latch.class`]

```
namespace std {
    class latch {
    public:
        constexpr explicit latch(ptrdiff_t expected);
        ~latch();

        latch(const latch&) = delete;
        latch& operator=(const latch&) = delete;

        void count_down(ptrdiff_t update = 1);
        bool try_wait() const noexcept;
        void wait() const;
        void arrive_and_wait(ptrdiff_t update = 1);

    private:
        ptrdiff_t counter; // exposition only
    };
}
```

- 1 A `latch` maintains an internal counter that is initialized when the `latch` is created. Threads can block on the `latch` object, waiting for `counter` to be decremented to zero.
- 2 Concurrent invocations of the member functions of `latch`, other than its destructor, do not introduce data races.

```
constexpr explicit latch(ptrdiff_t expected);
```

- 3 *Expects:* `expected >= 0` is true.
- 4 *Effects:* Initializes counter with `expected`.
- 5 *Throws:* Nothing.

```
void count_down(ptrdiff_t update = 1);
```

- 6 *Expects:* `update >= 0` is true, and `update <= counter` is true.
- 7 *Effects:* Atomically decrements `counter` by `update`. If `counter` is equal to zero, unblocks all threads blocked on `*this`.
- 8 *Synchronization:* Strongly happens before the returns from all calls that are unblocked.

9 *Throws:* `system_error` when an exception is required ([\[`thread.req.exception`\]](#)).

10 *Error conditions:* Any of the error conditions allowed for mutex types ([\[`thread.mutex.requirements.mutex`\]](#)).

```
bool try_wait() const noexcept;
```

11 *Returns:* With very low probability `false`. Otherwise `counter == 0`.

```
void wait() const;
```

12 *Effects:* If `counter` equals zero, returns immediately. Otherwise, blocks on `*this` until a call to `count_down` that decrements `counter` to zero.

13 *Throws:* `system_error` when an exception is required ([\[`thread.req.exception`\]](#)).

14 *Error conditions:* Any of the error conditions allowed for mutex types ([\[`thread.mutex.requirements.mutex`\]](#)).

```
void arrive_and_wait(ptrdiff_t update = 1);
```

15 *Effects:* Equivalent to:

```
    count_down(update);  
    wait();
```

32.◆.2 Barriers

[[thread.coord.barrier](#)]

- 1 A barrier is a thread coordination mechanism whose lifetime consists of a sequence of barrier phases, where each phase allows at most an expected number of threads to block until the expected number of threads arrive at the barrier. [*Note:* A barrier is useful for managing repeated tasks that are handled by multiple threads. - *end note*]

32.◆.2.1 Header `<barrier>` synopsis

[[thread.coord.barrier.syn](#)]

```
namespace std {  
    template<class CompletionFunction = see below>  
        class barrier;  
}
```

32.2.2 Class template **barrier**

[thread.coord.barrier.class]

```
namespace std {
    template<class CompletionFunction = see below>
    class barrier {
    public:
        using arrival_token = see below;

        constexpr explicit barrier(ptrdiff_t phase_count,
                                   CompletionFunction f = CompletionFunction(
~barrier());

        barrier(const barrier&) = delete;
        barrier& operator=(const barrier&) = delete;

        [[nodiscard]] arrival_token arrive(ptrdiff_t update = 1);
        void wait(arrival_token&& arrival) const;

        void arrive_and_wait();
        void arrive_and_drop();

    private:
        CompletionFunction completion; // exposition only
    };
}
```

1 Each **barrier phase** consists of the following steps:

- (1.1) — The expected count is decremented by each call to `arrive` or `arrive_and_drop`.
- (1.2) — When the expected count reaches zero, the phase completion step is run. For the specialization with the default value of the `CompletionFunction` template parameter, the completion step is run as part of the call to `arrive` or `arrive_and_drop` that caused the expected count to reach zero. For other specializations, the completion step is run on one of the threads that arrived at the barrier during the phase.
- (1.3) — When the completion step finishes, the expected count is reset to what was specified by the `expected` argument to the constructor, possibly adjusted by calls to `arrive_and_drop`, and the next phase starts.

- 2 Each phase defines a ***phase synchronization point***. Threads that arrive at the barrier during the phase can block on the phase synchronization point by calling `wait`, and will remain blocked until the phase completion step is run.
- 3 The ***phase completion step*** that is executed at the end of each phase has the following effects:
 - (3.1) — Invokes the completion function, equivalent to `completion()`.
 - (3.2) — Unblocks all threads that are blocked on the phase synchronization point.

The end of the completion step strongly happens before the returns from all calls that were unblocked by the completion step. For specializations that do not have the default value of the `CompletionFunction` template parameter, the behavior is undefined if any of the `barrier` object's member functions other than `wait` are called while the completion step is in progress.

- 4 Concurrent invocations of the member functions of `barrier`, other than its destructor, do not introduce data races. The member functions `arrive` and `arrive_and_drop` execute atomically.
- 5 `CompletionFunction` shall meet the *Cpp17MoveConstructible* requirements (Table [26](#)) and the *Cpp17Destructible* requirements (Table [30](#)).
`is_nothrow_invocable_v<CompletionFunction&>` shall be true.
- 6 The default value of the `CompletionFunction` template parameter is an unspecified type, such that, in addition to satisfying the requirements of `CompletionFunction`, it meets the *Cpp17DefaultConstructible* requirements (Table [25](#)) and `completion()` has no effects.
- 7 `barrier::arrival_token` is an unspecified type, such that it meets the *Cpp17MoveConstructible* (Table [26](#)), *Cpp17MoveAssignable* (Table [28](#)), and *Cpp17Destructible* (Table [30](#)) requirements.

```
constexpr explicit barrier(ptrdiff_t phase_count,  
                           CompletionFunction f = CompletionFunction());
```

- 8 *Expects:* `phase_count >= 0` is true.
- 9 *Effects:* Sets both the initial expected count for each barrier phase and the current expected count for the first phase to `phase_count`. Initializes `completion` with `std::move(f)`. Starts the first phase. [*Note:* If `phase_count` is 0 this object can only be destroyed. — *end note*]

10 *Throws:* Any exception thrown by CompletionFunction's move constructor.

```
[[nodiscard]] arrival_token arrive(ptrdiff_t update = 1);
```

11 *Expects:* `update > 0` is true, and `update` is less than or equal to the expected count for the current barrier phase.

12 *Effects:* Constructs an object of type `arrival_token` that is associated with the phase synchronization point for the current phase. Then, decrements the expected count by `update`.

13 *Synchronization:* The call to `arrive` strongly happens before the start of the phase completion step for the current phase.

14 *Returns:* The constructed `arrival_token` object.

15 *Throws:* `system_error` when an exception is required ([\[thread.req.exception\]](#)).

16 *Error conditions:* Any of the error conditions allowed for mutex types ([\[thread.mutex.requirements.mutex\]](#)).

17 [*Note:* This call can cause the completion step for the current phase to start. - *end note*]

```
void wait(arrival_token&& arrival) const;
```

18 *Expects:* `arrival` is associated with the phase synchronization point for the current phase or the immediately preceding phase of the same barrier object.

19 *Effects:* Blocks at the synchronization point associated with `std::move(arrival)` until the phase completion step of the synchronization point's phase is run. [*Note:* If `arrival` is associated with the synchronization point for a previous phase, the call returns immediately. - *end note*]

20 *Throws:* `system_error` when an exception is required ([\[thread.req.exception\]](#)).

21 *Error conditions:* Any of the error conditions allowed for mutex types ([\[thread.mutex.requirements.mutex\]](#)).

```
void arrive_and_wait();
```

22 *Effects:* Equivalent to: `wait(arrive())`.

```
void arrive_and_drop();
```

- 23 *Expects:* The expected count for the current barrier phase is greater than zero.
- 24 *Effects:* Decrements the initial expected count for all subsequent phases by one. Then decrements the expected count for the current phase by one.
- 25 *Synchronization:* The call to `arrive_and_drop` strongly happens before the start of the phase completion step for the current phase.
- 26 *Throws:* `system_error` when an exception is required ([\[`thread.req.exception`\]](#)).
- 27 *Error conditions:* Any of the error conditions allowed for mutex types ([\[`thread.mutex.requirements.mutex`\]](#)).
- 28 [*Note:* This call can cause the completion step for the current phase to start. - *end note*]

Create the following feature test macros with the given headers, adding them to the table in [support.limits.general]:

- `__cpp_lib_atomic_lock_free_type_aliases` in `<atomic>`, which implies that `atomic_signed_lock_free` and `atomic_unsigned_lock_free` types are available.
- `__cpp_lib_atomic_flag_test` in `<atomic>`, which implies the test methods and free functions for `atomic_flag` are available.
- `__cpp_lib_atomic_wait` in `<atomic>`, which implies the `wait`, `notify_one`, and `notify_all` methods and free functions for `atomic` and `atomic_flag` are available.
- `__cpp_lib_semaphore` in `<semaphore>`, which implies that `counting_semaphore` and `binary_semaphore` are available.
- `__cpp_lib_latch` in `<latch>`, which implies that `latch` is available.
- `__cpp_lib_barrier` in `<barrier>`, which implies that `barrier` is available.

§ Index

§ Terms defined by this specification

[atomic notifying operations](#), in §3

[Atomic waiting operations](#), in §3

[barrier phase](#), in §3

[eligible to be unblocked](#), in §3

[phase completion step](#), in §3

[phase synchronization point](#), in §3

§ References

§ Informative References

[N4810]

Richard Smith. [Working Draft, Standard for Programming Language C++](https://wg21.link/n4810). 15 March 2019. URL: <https://wg21.link/n4810>

[P0514R4]

Olivier Giroux. [Efficient concurrent waiting for C++20](https://wg21.link/p0514r4). 3 May 2018. URL: <https://wg21.link/p0514r4>

[P0666R2]

Olivier Giroux. [Revised Latches and Barriers for C++20](https://wg21.link/p0666r2). 6 May 2018. URL: <https://wg21.link/p0666r2>

[P0995R1]

JF Bastien, Olivier Giroux, Andrew Hunter. [Improving atomic_flag](https://wg21.link/p0995r1). 22 June 2018. URL: <https://wg21.link/p0995r1>

[P1258R0]

Detlef Vollmann. [Don't Make C++ Unimplementable On Small CPUs](https://wg21.link/p1258r0). 8 October 2018. URL: <https://wg21.link/p1258r0>

[P1633R0]

David Olsen, Olivier Giroux. [Amendments to the C++20 Synchronization Library](https://wg21.link/p1633r0). 16 June 2019. URL: <https://wg21.link/p1633r0>